

FlexWATCH AI SDK

Version 4.1.1

(Apr, 19, 2025)



SEYEON TECH Co., Ltd.

<http://www.flexwatch.co.kr>

Table of Contents

1. FlexWATCH AI SDK	3
1.1. Prepare PC	3
1.2. Creating a Python virtual environment using virtualenv	3
1.3. SSD, YOLO (Detector) training environment setting	3
1.4. Changes to AI threshold	5
1.5. Camera firmware version available for YOLO model	7
1.6. MobilenetV2 + SSDLite (Detector) training	7
1.7. Deformed yolov7-tiny (Detector) training	10
1.8. Convert the trained model to an onnx file for SSD	12
1.9. Validating the Trained Model for SSD.....	13
1.10. Converting onnx file to binary file usable on EN675 for SSD	14
1.11. Converting onnx file to binary file usable on EN675 for YOLO	19
1.12. Create an AI OEM Model (tar.gz) to upload to the EN675	24
1.13. Create an AI OEM Model that supports multi-step inference	35
1.14. How to delete AI OEM Model	47
1.15. Download and build the FlexWatch AI SDK.....	48
1.16. Create an AI OEM MODEL tar.gz file for multi-step inference	50
1.17. How to edit Pre Post Processing source file.....	54
1.18. How to check AI inference and Post Process logs	58
1.19. Korean font support through osd	59
1.20. UTF-8 code table for supported Korean osd	59
1.21. Decrypt function for encrypted custom ai model file	63
1.22. Appendix	64
1.23. Document Revision History	66

1. FlexWATCH AI SDK

This manual guides you through the entire process of utilizing the AI OEM model (the AI model learned by the user) used in the EN675 camera, from deep learning model training to uploading it to the camera and using it.

The latest AI SDK supports backward compatibility, but for various PrePostProcess functions and performance improvements, please implement the AI OEM model using the latest firmware and AI SDK possible.

1.1. Prepare PC

We recommend Ubuntu 20.04.

1.2. Creating a Python virtual environment using virtualenv

1. Install python3-virtualenv using apt (advanced packaging tool)

```
$sudo apt install python3-virtualenv
```

2. Create my-venv (Python virtual environment name) using virtualenv.

```
$virtualenv my-venv
```

3. Activate my-venv.

```
$source my-venv/bin/activate
```

4. When my-venv is activated, it will look like this:

```
(my-venv) training@training-WS-C246M-PRO-Series:~$
```

1.3. SSD, YOLO (Detector) training environment setting

1. Download FlexWatch_AI_SDK_4.1.1_yyyymmdd.zip.

URL : sdk.flexwatch.com

ID	fwrnd
----	-------

PASSWORD	flexwatch
----------	-----------

2. Unzip the FlexWatch_AI_SDK_4.1.1_yyyymmdd.zip file.
3. Move the trainer_sy_yolo.tar file required for deep learning training to the prepared Ubuntu PC.
4. Unpack en675_trainer_sy.tar using the command below.

```
$tar -xvf en675_trainer_sy.tar
```

result:

total 28

drwxrwxr-x 6 aiserver aiserver 4096 Apr 18 14:26 ./

drwxrwxr-x 3 aiserver aiserver 4096 Apr 19 08:53 ../

drwxrwxr-x 10 aiserver aiserver 4096 Mar 11 10:58 mobilenetV2_SSDliteV1/

drwxrwxr-x 10 aiserver aiserver 4096 Mar 11 10:58 mobilenetV2_SSDliteV2/

drwxrwxr-x 12 aiserver aiserver 4096 Apr 18 15:28 mobilenetV2_SSDliteV2_full_rfb/

-rwxrw-r-- 1 aiserver aiserver 3053 Mar 11 08:49 npu_trainer_v09.yaml*

drwxrwxr-x 8 aiserver aiserver 4096 Mar 11 08:49 yolov7_add_rfb/

5. Go to the trainer directory using the command below.

```
$cd trainer
```

6. Deep Learning Environment Requirements

- A. Ubuntu 20.04
- B. Python 3.8
- C. Pytorch >= 1.7

7. Install anaconda (refer to the Internet)

8. Get and create a Conda environment for training (yaml file is in the trainer directory)

```
$conda env create -f npu_trainer_v{NPU SDK version}.yaml
```

9. Activate the conda virtual environment

```
$conda activate npu_trainer_v{NPU SDK version}
```

10. You need to reinstall Torch according to the CUDA version supported by your graphics card. If you do not have a graphics card, you can install Torch CPU version.

Download site: <https://pytorch.org/get-started/locally>

1.4. Changes to AI threshold

1. When compiling an AI model, <*.bin, *.cfg> are generated as a pair.
2. Example of cfg file

SSD model_cfg.cfg	YOLO model_cfg.cfg
OPERATION=SSD	OPERATION=YOLO
WIDTH=320	WIDTH=832
HEIGHT=320	HEIGHT=480
CONF_TH=0.3	SCORE_TH=0.3
IOU_TH=0.6	CONF_TH=0.3
NUM_CLASSES=21	IOU_TH=0.6
C000_LABEL_N_TH=BACKGROUND, 120	NUM_CLASSES=80
C001_LABEL_N_TH=aeroplane, 120	...
C002_LABEL_N_TH=bicycle, 120	C000_LABEL_N_TH=person, 120
C003_LABEL_N_TH=bird, 120	C001_LABEL_N_TH=bicycle, 120
C004_LABEL_N_TH=boat, 120	C002_LABEL_N_TH=car, 120
	C003_LABEL_N_TH=motorcycle, 120

3. Difference between the threshold in the model_cfg.cfg file and the threshold in the WebAdmin page

Threshold of Model_cfg.cfg	Threshold of WebAdmin
70	40

When the threshold is as above, a larger threshold (here 70) becomes the threshold that actually has an effect. If you want to set a threshold for each class, set the threshold (range 10~99%) of the Seyeon Tech camera Webadmin to the lowest value 10% and then adjust the threshold in model_cfg.cfg. If you want to apply the same threshold to all classes, set the threshold value of model_cfg.cfg to 0 and set the threshold in WebAdmin.

< Threshold setting screen of Seyeon Tech Camera WebAdmin >

Object Min Confidence	40	(10 ~ 100 %)
Object Min Width	1	(1 ~ 100 %)
Object Max Width	100	(1 ~ 100 %)
Object Min Height	1	(1 ~ 100 %)
Object Max Height	100	(1 ~ 100 %)
Object Tracking Enable	☒ Enable ☐ Disable	
Objects Display on Raw Video	☐ Enable ☒ Disable	
Objects Display on Viewer	☒ Enable ☐ Disable	
Synchronize video and BBox	☐ Enable ☒ Disable	
Letter Box Enable/Disable	☒ Enable ☐ Disable	
Crop Offset	5	(-100~100)
Crop from the copied original UYV	☐ Enable ☒ Disable	

1.5. Camera firmware version available for YOLO model

1. Firmware classification based on availability of Yolo model

Yolo is not available	rf_ec-4.63-040-rv.tar.gz 까지
Yolo available	rf_ec-4.63-500-rv.tar.gz 이후

1.6. MobilenetV2 + SSDLite (Detector) training

2. Preparing the dataset

Download the VOC 2007, 2012 datasets from the links below.

<https://drive.google.com/file/d/12RmxmsmwN-u-UyWf2bLkEBRTyoCy6-IT/view?usp=sharing>

3. Unzip the downloaded dataset.

```
$cd /home/training/trainer/mobilenetV2_SSDliteV2_full_rfb
```

```
$unzip PASCAL_VOC_Dataset.zip
```

Result :

```
(my-venv) training@training-WS-C246M-PRO-
```

```
Series:~/trainer/mobilenetV2_SSDliteV2_full_rfb$ ll PASCAL_VOC_Datasettotal 16
```

```
drwxrwxr-x  4 training training 4096 Feb  2 08:29 ./
```

```
drwxrwxr-x 11 training training 4096 Feb  2 08:28 ../
```

```
drwxrwxr-x  5 training training 4096 May  3 2021 PASCAL_VOC_2007/
```

```
drwxrwxr-x  4 training training 4096 May 17 2021 PASCAL_VOC_2012/
```

4. Deep learning progress

```
$ python3 detector_train.py --
```

```
dataset_root ./PASCAL_VOC_Dataset/PASCAL_VOC_2012/ --model ssdlite512 --
```

```
batch_size 16 --dataset_domains train,validation --epochs 200 --
```

```
class_path ./PASCAL_VOC.txt
```

```

result:
(my-venv) training@training-WS-C246M-PRO-
Series:~/trainer/mobilenetV2_SSDliteV2_full_rfb$ python3 detector_train.py --
dataset_root ./PASCAL_VOC_Dataset/PASCAL_VOC_2012 --model ssdlite512 --
batch_size 16 --dataset_domains train,validation --epochs 200 --
class_path ./PASCAL_VOC.txt
Start Training
Model: ssdlite512
Epochs : 200
Batch Size: 16
Learning Rate: 0.01000
Optimizer : AdamW
Scheduler: cosine

Downloading: "https://download.pytorch.org/models/mobilenet_v2-b0353104.pth" to
/home/training/.cache/torch/hub/checkpoints/mobilenet_v2-b0353104.pth
100.0%
2023-02-02 11:34:34.670912 epoch#0 start
[32 / 11540] - loss=14.548

```

5. Check the trained model file (the model file is saved in the ./checkpoints directory every 5 epochs.)

```
$ll ./checkpoints
```

```

result:
(my-venv) training@training-WS-C246M-PRO-
Series:~/trainer/mobilenetV2_SSDliteV2_full_rfb $ ll checkpoints/
total 88545760
drwxrwxr-x  2 training training  49152 Jan 26 11:01 ./
drwxrwxr-x 13 training training   4096 Jan 19 15:00 ../

```


FlexWATCH AI SDK

```
-rw-rw-r-- 1 training training 92759334 Dec 23 09:50 ssdlite512_obj_10.pth  
-rw-rw-r-- 1 training training 92759334 Dec 23 10:48 ssdlite512_obj_15.pth  
-rw-rw-r-- 1 training training 92759334 Dec 23 11:46 ssdlite512_obj_20.pth  
-rw-rw-r-- 1 training training 92759334 Dec 23 12:45 ssdlite512_obj_25.pth
```

How to enable the Letterbox option (option to maintain the aspect ratio of objects during learning)

```
--enable_letterbox True
```

1.7. Deformed yolov7-tiny (Detector) training

This is not the original yolov7-tiny model. This is a yolov7-tiny model with the SPP block modified to an RFB block because the EN675 NPU does not support the SPP block.

deformed yolov7-tiny performance

👉 Train Dataset : COCO 2017 (train) 👉 Test Dataset : COCO 2017 (val)

Model	mAP at IOU = 0.50:0.95 (GPU)	mAP at IOU = 0.50:0.95 (NPU)	mAP at IOU = 0.50 (GPU)	mAP at IOU = 0.50 (NPU)
640_352_eyenix_model_v2	33.4	27.5	49.9	44.9
832_480_eyenix_model_v2	36.9	29.1	54.8	48.2

1. Prepare the dataset

Annotation (.txt) file configuration Format :

Class_number, x_center(0~1), y_center(0~1), width(0~1), height(0~1)

Users need to structure their user dataset for training or inference as follows:

FlexWATCH AI SDK

```
dataset/  
  images/  
    train/  
      *.jpg  
    val/  
      *.jpg  
    test/  
      *.jpg  
  
  labels/  
    train/  
      *.txt  
    val/  
      *.txt  
    test/  
      *.txt
```

Create a yaml file for your dataset

```
# dataset path  
train: ./dataset/images/train/  
val: ./dataset/images/val/  
test: ./dataset/images/test/  
  
# number of classes  
nc: 1  
  
# class names  
names: [ 'person' ]
```

2. Deep learning progress

The example.yaml used here is literally an example dataset. The user should use the yaml file for information about the dataset they actually want to apply.

```
CUDA_VISIBLE_DEVICES=0 python3 train.py --batch-size 32 --data  
data/example.yaml --img-size-train 640 640 --img-size-test 640 640 --cfg cfg/yolov7-  
tiny-silu-en675-rfb.yaml --weights yolov7-tiny.pt --hyp data/hyp.scratch.tiny.yaml
```

3. Inference

Example of running inference

- The trained pt files are stored for each experience in the "runs" directory.

FlexWATCH AI SDK

```
CUDA_VISIBLE_DEVICES=0 python3 detect.py --weights {pt file} --conf 0.25 --img-size 640 640 --source dataset/images/test/000275.jp
```

Test

Running example during testing

- The learned pt files are stored for each experience in the "runs" directory.

```
CUDA_VISIBLE_DEVICES=0 python3 test.py --data data/example.yaml --img 640 640 --batch-size 32 --conf 0.001 --iou 0.65 --weights {pt file}
```

4. Convert to ONNX

Example of running onnx export - trained pt files are saved in the "runs" directory by exp.

```
python3 export.py --weights {pt file} --img-size 640 640 --simplify --batch-size 1
```

The onnx and anchor files will be created in the directory where the pt files were.
The onnx and anchor files are required.

1.8. Convert the trained model to an onnx file for SSD

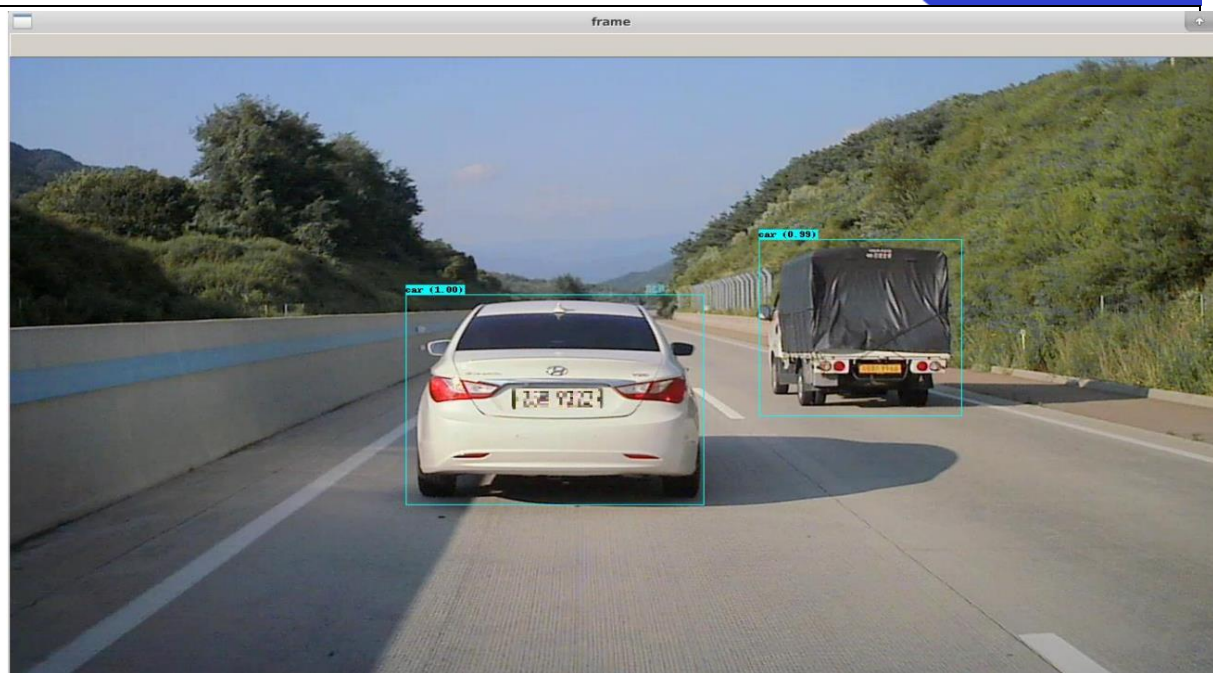
11. Modify line 112 of detector_single.py as follows.

```
torch.onnx.export(model, x, filename, opset_version=9)
```

12. The inference results can be checked and converted to an .onnx file using the command below.

```
$python3 detector_single.py --model ssdlite512 --class_path ./PASCAL_VOC.txt --weight checkpoints/ssdlite512_obj_latest.pth --export { Image path to be inferred }
```

result:



(my-venv) training@training-WS-C246M-PRO-

Series:~/trainer/mobilenetV2_SSDliteV2_full_rfb \$ ll ssdlite*

total 88545760

drwxrwxr-x 2 training training 49152 Jan 26 11:01 ./

drwxrwxr-x 13 training training 4096 Jan 19 15:00 ../

-rw-rw-r-- 1 training training 92759334 Dec 23 09:50 ssdlite512.onnx

-rw-rw-r-- 1 training training 92759334 Dec 23 10:48 ssdlite512.anchor

1.9. Validating the Trained Model for SSD

1. You can verify the learned model and check the mAP (mean Average Precision) using the command below. (The range of mAP is 0.0~1.0. If mAP is too low, you need to modify the dataset and hyperparameters and re-learn.)

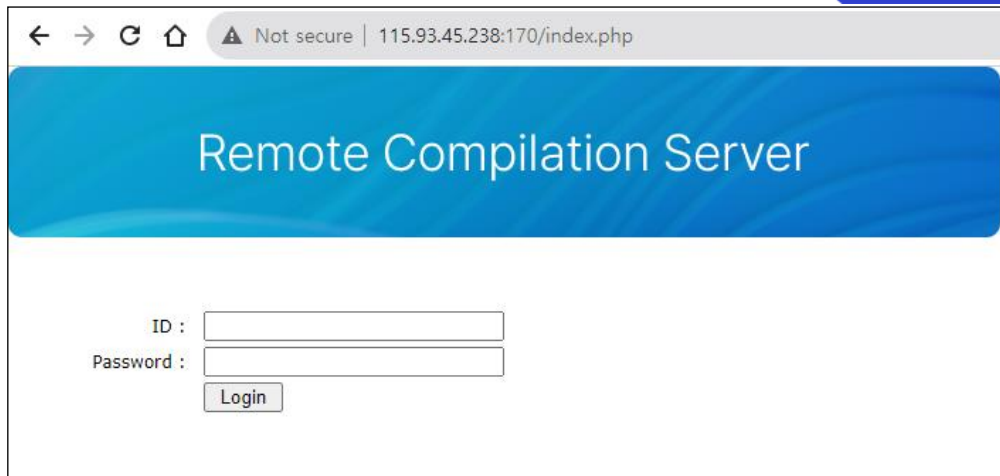
```
$python3 detector_eval.py --
dataset_root ./PASCAL_VOC_Dataset/PASCAL_VOC_2007/ --dataset_domains test --
model ssdlite512 --class_path ./PASCAL_VOC.txt --weight
checkpoints/ssdlite512_obj_latest.pth
result:
```

FlexWATCH AI SDK

```
AP(aeroplane) = 0.361
AP(bicycle) = 0.202
AP(bird) = 0.117
AP(boat) = 0.022
AP(bottle) = 0.000
AP(bus) = 0.238
AP(car) = 0.406
AP(cat) = 0.276
AP(chair) = 0.029
AP(cow) = 0.172
AP(diningtable) = 0.113
AP(dog) = 0.254
AP(horse) = 0.265
AP(motorbike) = 0.174
AP(person) = 0.447
AP(pottedplant) = 0.004
AP(sheep) = 0.070
AP(sofa) = 0.132
AP(train) = 0.162
AP(tvmonitor) = 0.085
mAP = 0.177
```

1.10. Converting onnx file to binary file usable on EN675 for SSD

1. Supports input resolutions of 320, 416, 512, and 640.
2. Prepare four files: the .onnx and .anchor files created above, the label.txt file used for learning, and sample_dataset.zip (compress about 100 data sets used for learning).
3. Access <http://115.93.45.238:170>.
4. The following webpage appears.



← → ↻ 🏠 ⚠ Not secure | 115.93.45.238:170/index.php

Remote Compilation Server

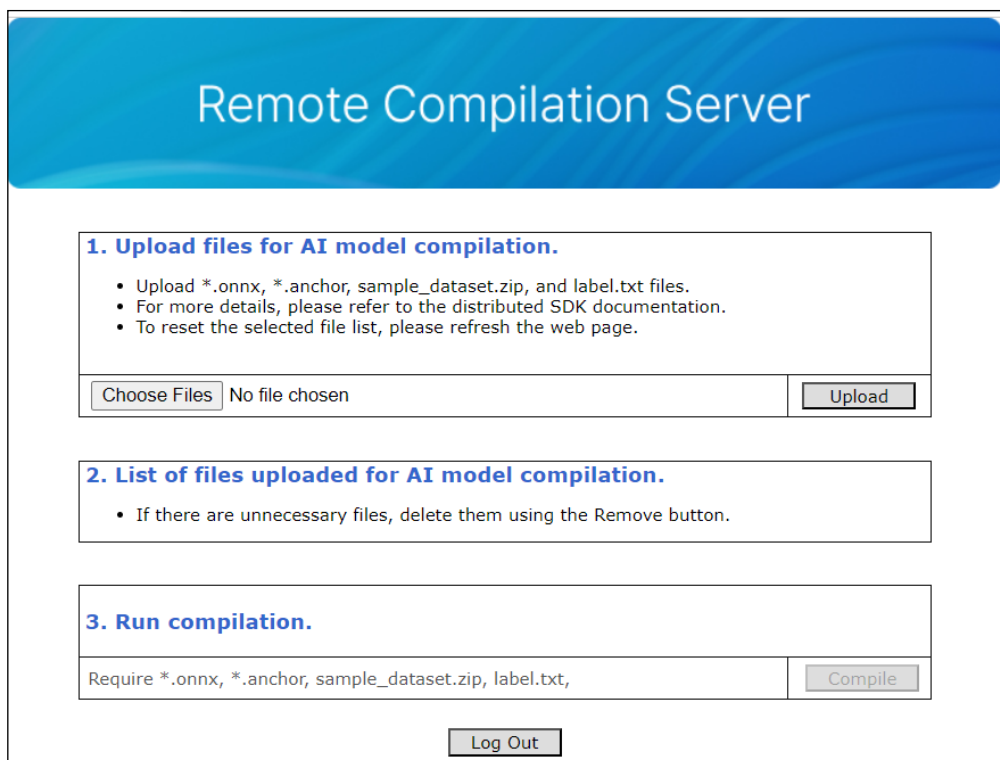
ID :

Password :

Please log in using the login information below.

id	manager
pw	manager

5. A web page like the one below will appear.



Remote Compilation Server

1. Upload files for AI model compilation.

- Upload *.onnx, *.anchor, sample_dataset.zip, and label.txt files.
- For more details, please refer to the distributed SDK documentation.
- To reset the selected file list, please refresh the web page.

No file chosen

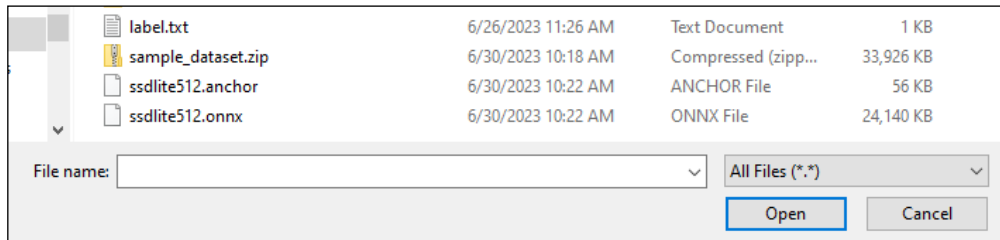
2. List of files uploaded for AI model compilation.

- If there are unnecessary files, delete them using the Remove button.

3. Run compilation.

Require *.onnx, *.anchor, sample_dataset.zip, label.txt,

6. In the Upload files for AI model compilation section, click the Choose Files button and upload *.onnx, *.anchor, sample_dataset.zip, and label.txt files.



1. Upload files for AI model compilation.

- Upload *.onnx, *.anchor, sample_dataset.zip, and label.txt files.
- For more details, please refer to the distributed SDK documentation.
- To reset the selected file list, please refresh the web page.



7. Once the upload is complete, you can see the Compile button activated along with the list of uploaded files as shown below. Click the Compile button to proceed with compilation.

Remote Compilation Server

1. Upload files for AI model compilation.

- Upload *.onnx, *.anchor, sample_dataset.zip, and label.txt files.
- For more details, please refer to the distributed SDK documentation.
- To reset the selected file list, please refresh the web page.

No file chosen

2. List of files uploaded for AI model compilation.

- If there are unnecessary files, delete them using the Remove button.

./label.txt	Remove
./sample_dataset.zip	Remove
./ssdlite512.anchor	Remove
./ssdlite512.onnx	Remove

3. Run compilation.

Ready to Compile AI

8. You can check the compilation progress with the progress bar. Depending on the model, compilation usually takes less than 10 minutes.

Remote Compilation Server

- Compilation will be completed in about 10 minutes.

Compiling...

9. Compilation is complete. The ai model binaries that can be used in EN675 are

highlighted in blue. You can download them and use them in EN675. If necessary, you can also download the compilation log. For security reasons, please be sure to delete the uploaded files. The image below shows the result of compiling the object detection model (Einix Reference) with a 512x512 input image size trained using Pascal VOC 2012.

Remote Compilation Server

4. Download results and delete uploaded files

- Please download result.bin, compile_log.txt, compile_result.txt.
- Please delete all files to prevent leakage of your AI model data.
- If you have deleted all files, be sure to log out using the log out button for security reasons.

./compile_log.txt	Remove	Download
./compile_result.txt	Remove	Download
./label.txt	Remove	
./result.bin	Remove	Download
./sample_dataset.zip	Remove	
./ssdlite512.anchor	Remove	
./ssdlite512.onnx	Remove	

Notice • After completing use, Please click the "Log Out" button.
The page will automatically Log Out after 5 minutes.
To compile again, click the File Upload Page button to return to the File Upload Page.

[File Upload Page](#) [Log Out](#)

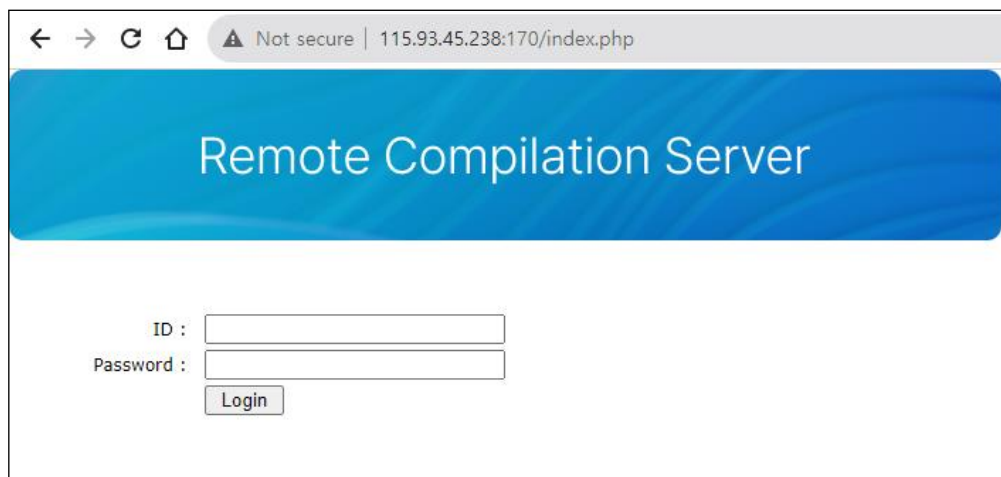
10. After downloading the result file and deleting the uploaded file, click the Logout button to log out. If you want to compile again, you can go back to the File Upload Page by clicking the File Upload Page button.

Notice • After completing use, Please click the "Log Out" button.
The page will automatically Log Out after 5 minutes.
To compile again, click the File Upload Page button to return to the File Upload Page.

[File Upload Page](#) [Log Out](#)

1.11. Converting onnx file to binary file usable on EN675 for YOLO

1. Prepare four files: the .onnx and .anchor files created for YOLO above, the coco.label file used for training, and sample_dataset.zip (compress about 100 data sets used for training).
2. Go to <http://115.93.45.238:170>.
3. The following webpage will appear.



Remote Compilation Server

ID :

Password :

Login

Please log in using the login information below.

id	manager
pw	manager

4. When the web page below appears, select YOLO.

The top screenshot shows a web browser at the URL `115.93.45.238:170/build_select.php`. The page has a blue header with the text "Remote Compilation Server". Below the header, there are two buttons: "SSD" and "YOLO". The "YOLO" button is highlighted with a green rectangular box. Below these buttons is a "Log Out" button.

The bottom screenshot shows a web browser at the URL `115.93.45.238:170/file_upld2.php`. The page has a blue header with the text "Remote Compilation Server". Below the header, there are three sections:

- 1. Upload files for AI model compilation.**
 - Upload *.onnx, *.anchor, sample_image.zip, and coco.label files.
 - For more details, please refer to the distributed SDK documentation.
 - To reset the selected file list, please refresh the web page.

Below the instructions, there is a button labeled "파일 선택" (File Select) and the text "선택된 파일 없음" (No files selected). To the right of this is an "Upload" button.
- 2. List of files uploaded for AI model compilation.**
 - If there are unnecessary files, delete them using the Remove button.

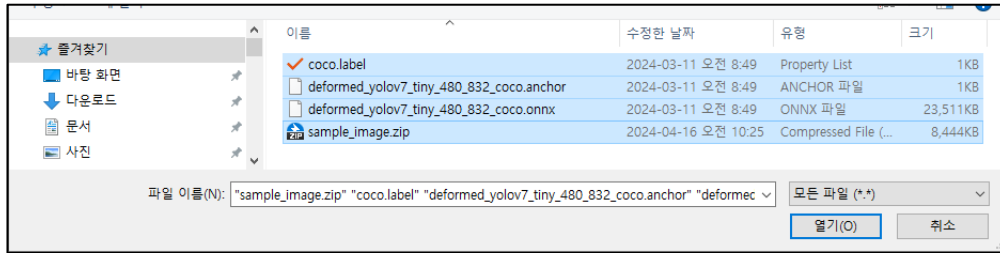
./clean_up.sh	Remove
./npu_compiler_v09.yaml	Remove
- 3. Run compilation.**

Require *.onnx, *.anchor, sample_image.zip, coco.label,

Below this is a "Compile" button.

At the bottom of the page is a "Log Out" button.

5. In the Upload files for AI model compilation section, click the Choose Files button and upload *.onnx, *.anchor, sample_image.zip, and coco.label files.



1. Upload files for AI model compilation.

- Upload *.onnx, *.anchor, sample_dataset.zip, and label.txt files.
- For more details, please refer to the distributed SDK documentation.
- To reset the selected file list, please refresh the web page.



6. Once the upload is complete, you can see the Compile button activated along with the uploaded file list as shown below. Click the Compile button to proceed with compilation.

Remote Compilation Server

1. Upload files for AI model compilation.

- Upload *.onnx, *.anchor, sample_image.zip, and coco.label files.
- For more details, please refer to the distributed SDK documentation.
- To reset the selected file list, please refresh the web page.

파일 선택

선택된 파일 없음

Upload

2. List of files uploaded for AI model compilation.

- If there are unnecessary files, delete them using the Remove button.

./clean_up.sh	Remove
./coco.label	Remove
./deformed_yolov7_tiny_480_832_coco.anchor	Remove
./deformed_yolov7_tiny_480_832_coco.onnx	Remove
./npu_compiler_v09.yaml	Remove
./sample_image.zip	Remove

3. Run compilation.

Ready to Compile AI

Compile

Log Out

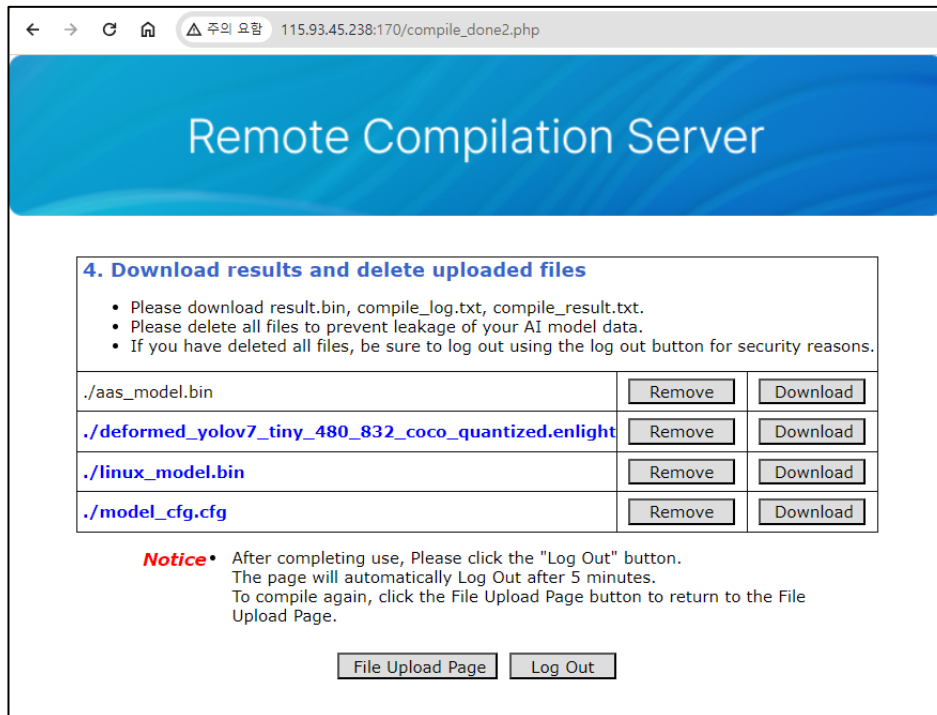
7. Progress bar display is under development (currently it displays from 0% to 100% as soon as it is completed). Depending on the model, compilation usually takes less than 10 minutes.

Remote Compilation Server

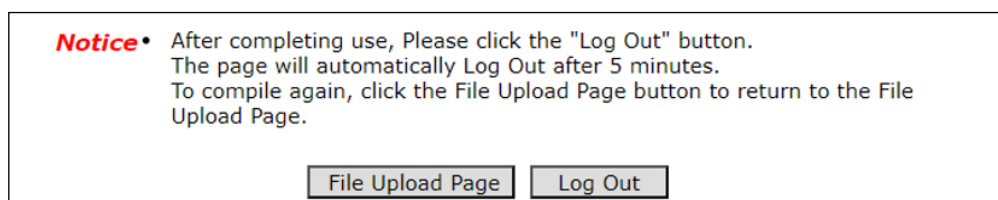
- Compilation will be completed in about 10 minutes.

Compiling...

8. Compilation is complete. The ai model binaries available for EN675 are highlighted in blue. You can download them and use them for EN675. For security reasons, please be sure to delete the uploaded files. The image below shows the result of compiling the deformed yolov7 model (inix reference) trained using the coco dataset.



9. After downloading the result file and deleting the uploaded file, click the Logout button to log out. If you want to compile again, you can go back to the File Upload Page by clicking the File Upload Page button.



1.12. Create an AI OEM Model (tar.gz) to upload to the EN675

1. Prepare the result.bin, result.cfg, and label.txt files created through the above process.
2. Change the names of result.bin and result.cfg to use in IP cameras.

Name rule: 248_<AiModelName>_0_<Learning resolution width>_<Learning resolution height>.bin

248_<AiModelName>_0_<Learning resolution width>_<Learning resolution height>.cfg

'_'(underscore) is a reserved character as a separator.

Do not insert '_'(underscore) in any position or between AiModelName.

When a model is uploaded with an incorrect name rule, you must connect to Telnet and delete one. Please refer to Appendix 1.22.1 for the Telnet connection method. AI model file path: /opt/nss/ai

If the input resolution changes during training of the AI Model used by the IP camera for inference, a reboot is required. (Ex: 672x384 -> 512x288)

(This manual uses 248_od6class_0_672_384.bin, 248_od6class_0_672_384.cfg.)

This file name rule is limited to single model inference, and the name will be different in case of multi-stage inference. For multi-stage inference, refer to 1.13 of this document.)

Depending on the AI model and post-process results, the prefix of the AI model file must be distinguished as follows.

Prefix	AI model distinction	PrePostProcess function name and calling order
248_	Single model	1. PrePostProcess4Decryption(.bin) 2. PrePostProcess4Decryption(.cfg) 3. PrePostProcess4Init 4. PostProcess4MakeCustomObjResult 5. PostProcess4MakeObjResultFinalize

6. PrePostProcess4Term

3. Change the label.txt file name to the same as the binary name changed above, change the file extension to .tab, and change the contents as follows.

label.txt (Before change)	248_od6class.tab (After change)
aeroplane	1=aeroplane
bicycle	2=bicycle
bird	3=bird
boat	4=boat
...	...

The line breaks in .tab files must be Linux-style. The line breaks used by each operating system are as follows::

Operating System	Line break character
Windows	\r\n (Carriage Return + Line Feed)
Unix/Linux	\n (Line Feed)
macOS	\r (Carriage Return)

You can check the line break style of a text file by using the file command in Linux.

```
$file text.tab
text.tab: ASCII text, with CRLF line terminators
```

If it is not in the Unix/Linux style as above, change the line break character in the .tab file using a tool that changes it to a Unix/Linux style line break.

```
$sudo apt install dos2unix
$dos2unix text.tab
```

```
dos2unix: converting file text.tab to Unix format...
```

Once the required files are ready, compress them with the command below.
Build libPrePostProcess.so.1.0.0 by referring to 1.10 of this manual.

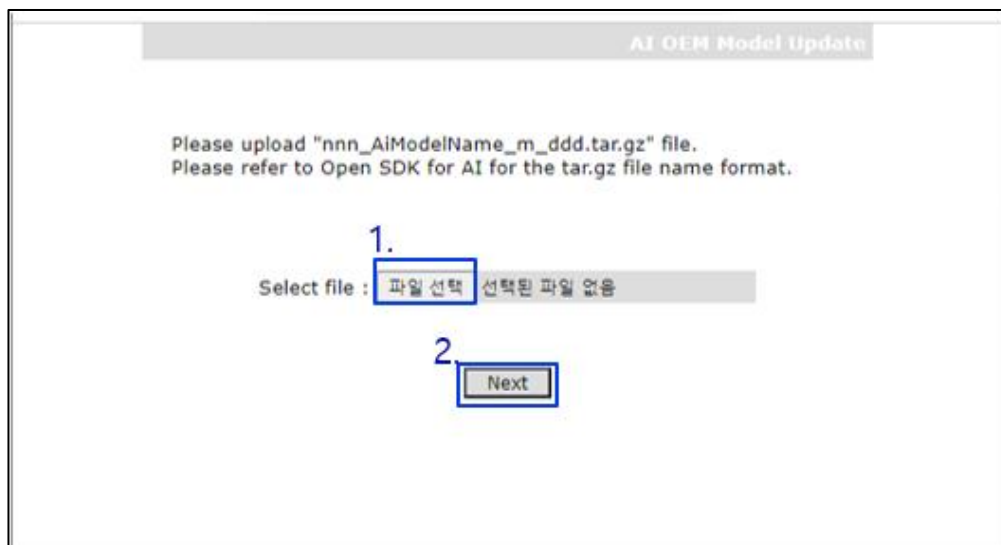
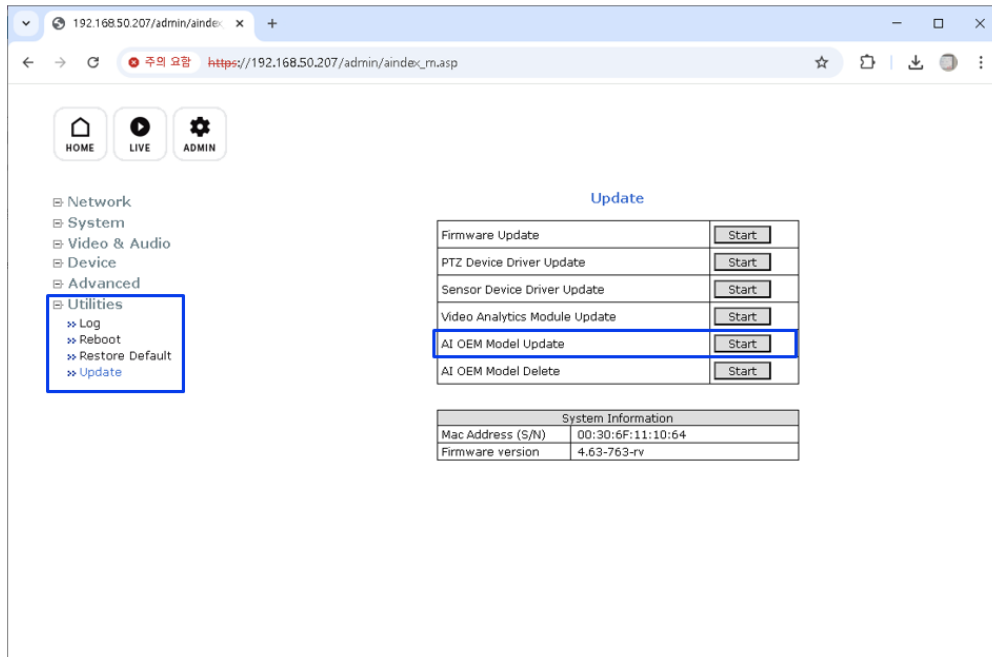
Compress tar.gz on Linux OS.

Compress firmware rf_ec-4.63-760-rv.tar.gz and later versions as follows.

```
tar -zcvf 248_od6class_1_672_384.tar.gz \  
248_od6class_0_672_384.bin \  
248_od6class_0_672_384.cfg \  
248_od6class.tab \  
libPrePostProcess.so.1.0.0
```

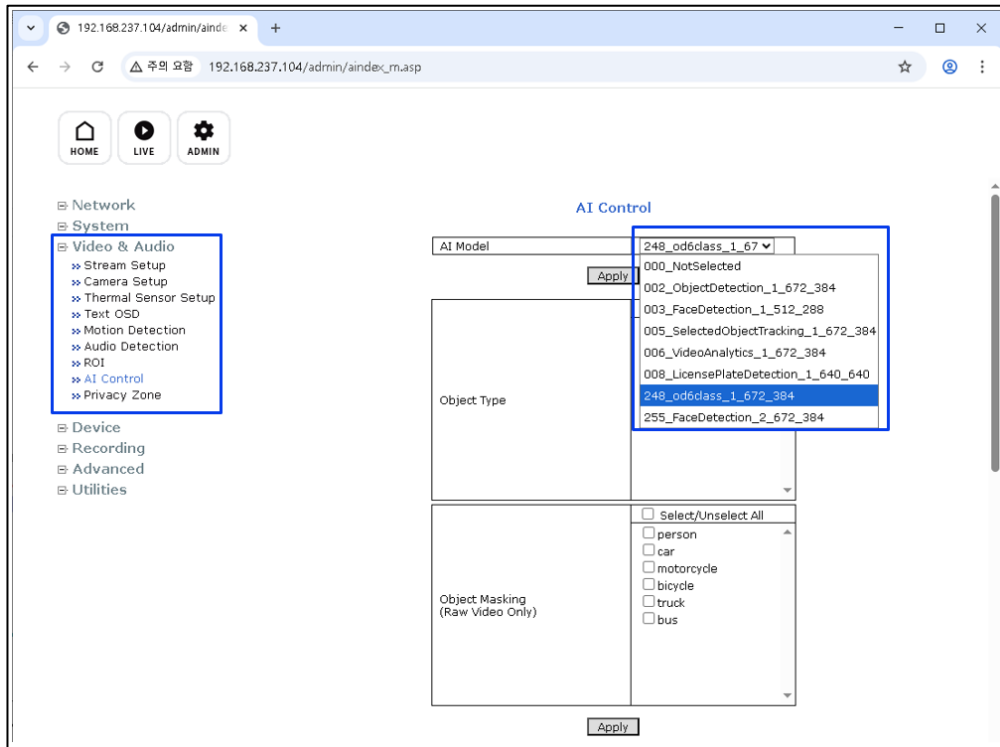
For firmware versions prior to rf_ec-4.63-758-rv.tar.gz, please refer to T4048-10-FlexWATCH_AI_SDK_4.1.0_Kor.pdf.

4. Upload the 248_od6class_1_672_384.tar.gz file as shown below.
Webadmin -> Utilities -> Update -> AI OEM Model Update

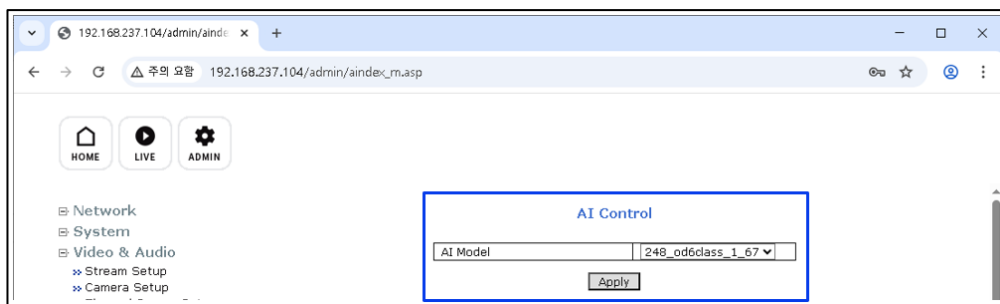


5. You can check the uploaded 248_od6class_1_672_384 model as shown in the figure below.

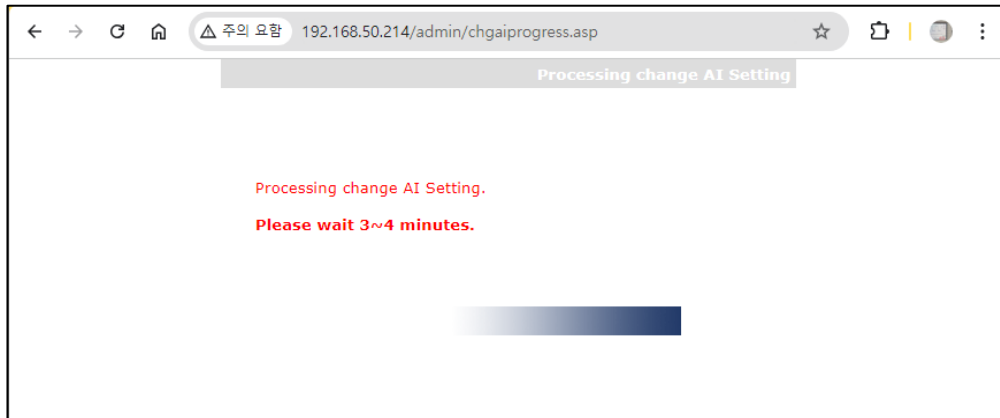
Webadmin -> Video & Audio -> AI Control



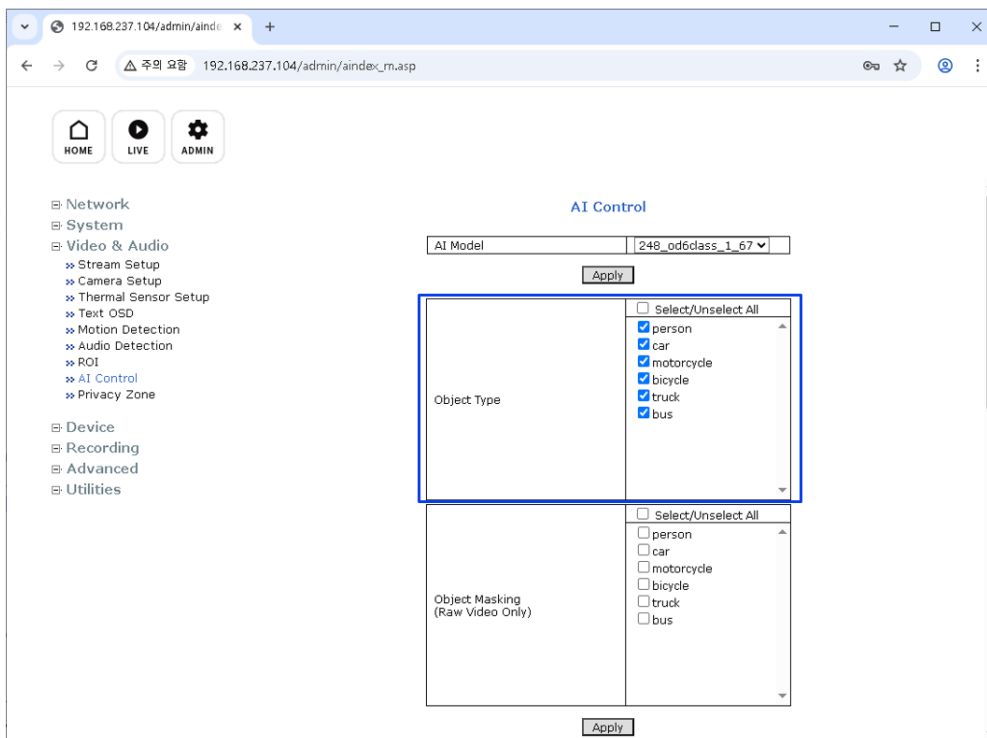
You can check the updated '248_od6class_1_672_384' model in the AI Model drop-down box. Select the '248_od6class_1_672_384' model and press the 'Apply' button.



It takes about 3 to 4 minutes for the AI model to be applied.

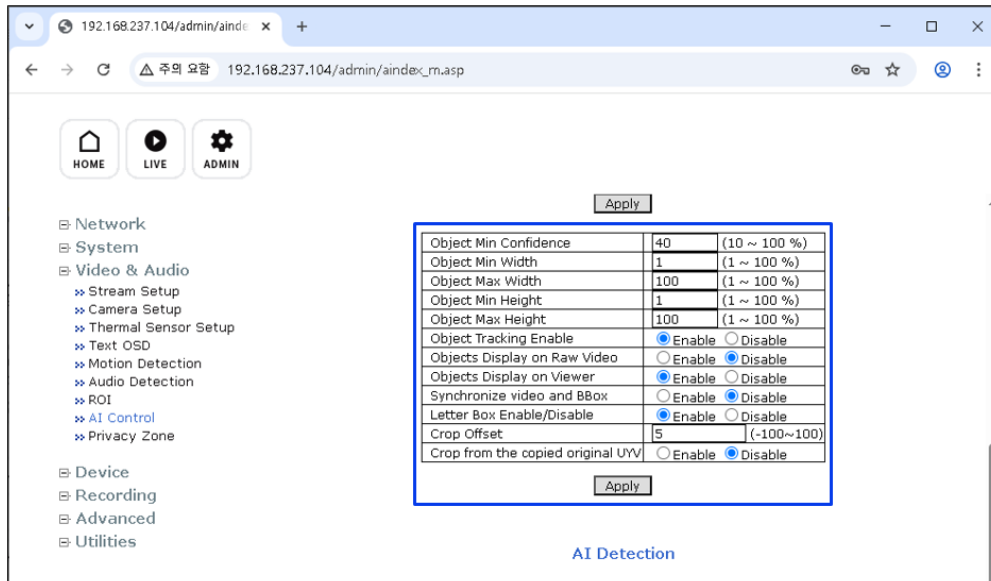


A reboot may be required if the inference resolution changes.



You can check the Object Type corresponding to the 248_od6class_1_672_384 model. Select the checkbox of the object you want to detect and press the 'Apply' button.

The AI option setting screen is as follows.



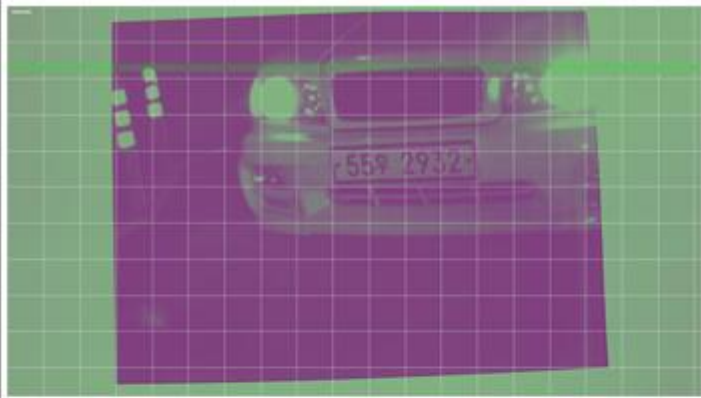
Here's a description of each AI option.

Menu	explanation
Object Type	Select the Object Type to display. 1) Person: Recognize and display a person. 2) Car: Recognize and display a car. 3) Motorcycle: Recognize and display a motorcycle. 4) Bicycle: Recognize and display a bicycle. 5) Truck: Recognize and display a truck. 6) Bus: Recognize and display a bus. * Multiple objects can be selected.
Object Masking (Raw Video Only)	The "Object Masking" function automatically detects and masks specific objects (etc.) in the video. This can protect personal information and prevent exposure of sensitive information. Select the Object Type to be masked.

	1) Person: Masks a person. 2) Car: Masks a car. 3) Motorcycle: Masks a motorcycle. 4) Bicycle: Masks a bicycle. 5) Truck: Masks a truck. 6) Bus: Masks a bus. * Multiple objects can be selected.
Object Min Confidence	You can set the reliability of the object. The standard is %. You can reduce errors by adjusting this value. If you set it to 70, it will be displayed when the reliability of the set object is 70% or higher. (The minimum value is 10.)
Object Min Width	Sets the minimum width of the object. (Minimum value is 1.) The size of the recognition area will be set in proportion to the original image size.
Object Max Width	Sets the maximum width of the object. (The maximum is 100.) The size of the recognition area will be set in proportion to the original image size.
Object Min Height	Sets the minimum height of the object. (Minimum value is 1.) The size of the recognition area will be set in proportion to the original image size.
Object Max Height	Sets the maximum height of the object. (The maximum is 100.) The size of the recognition area is set in proportion to the original image size.
Object Tracking Enable	Enables object tracking.
Object Display on Raw Video	This is a function that displays the recognized object as a bounding box in the original image when the camera transmits the video. To use the Object Masking (Raw Video Only) function,

	you must activate it. You can use this function in various clients (NVR, VLC, etc.).
Object Display on Viewer	Ability to display or hide objects recognized in the camera's web viewer.
Synchronize video and BBox	This feature synchronizes the video and bounding box. This option is only used when receiving video through fwstreamcgi of Seyontech SDK.
Letter Box Enable/Disable	This is a menu for selecting whether to use the aspect ratio when detecting an object. If you trained while maintaining the aspect ratio of the object using the Letter Box option, this is a menu for selecting whether to use this function on the camera.
Crop Offset	This is a function to set the margin range of the inference result (bounding box) of the first inference model when using a multi-stage inference model. Please refer to the image below.

Original Image

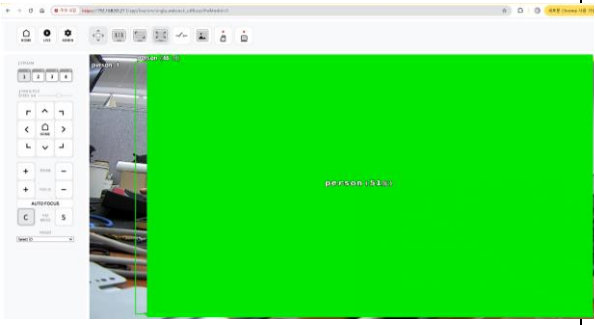
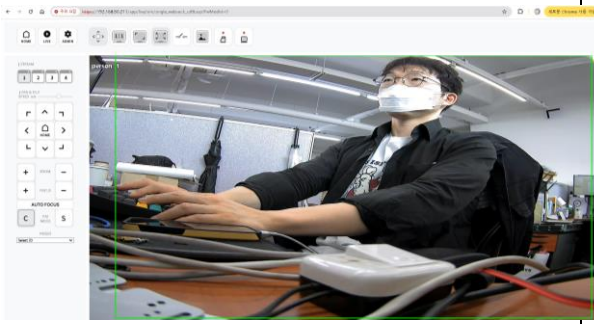


The green vertical lines in the image below are drawn every 5 pixels to visually indicate crop offset. Please not

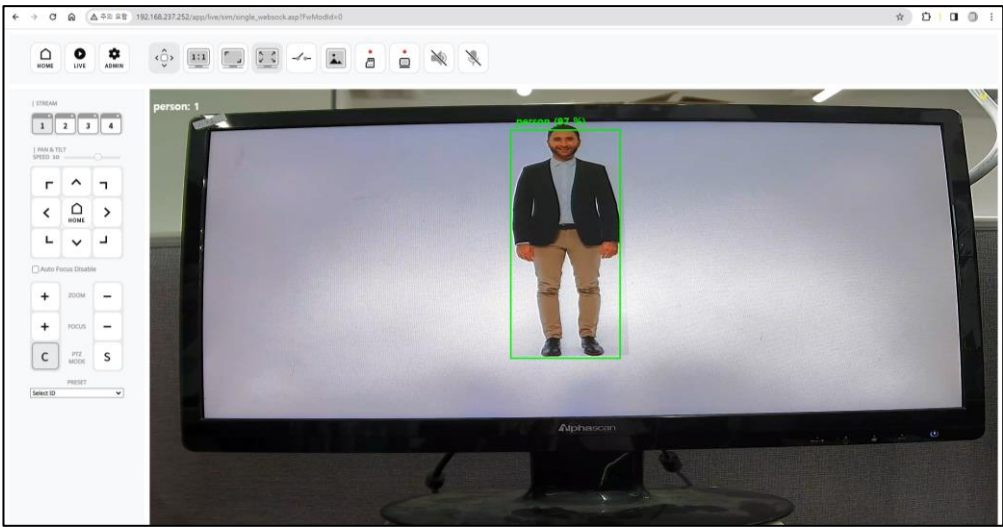


Example of Object Masking Screen)

The "Object Masking" function is a function that automatically detects and masks specific objects in the video. This can protect personal information and prevent exposure of sensitive information.

Apply masking	
Masking not applied	

6. You can check whether AI is working through Liveview.



1.13. Create an AI OEM Model that supports multi-step inference

The maximum number of AI models that can be used for multi-stage inference is 10. The maximum number may be limited depending on the amount of memory used by the AI model.

Depending on the AI model and the post-process result type, the prefix of the AI model file must be distinguished as follows.

Prefix	AI model distinction	PrePostProcess function name and calling order
255_	Single model	Deprecated for new implementations. Exists only for backward compatibility. For object detection, Object Detection, please use model 248_.
254_	When it is a multi-stage inference model and the length of the string resulting from the post process is less than 15	Deprecated for new implementations. Exists only for backward compatibility. For String Detection, please use model 247_.

	bytes including the NULL character (Post process result type: String)	
253_	Multi-stage inference model (Post process result type: Object)	Deprecated for new implementations. Exists only for backward compatibility. For object detection, please use model 248_.
252_	User Video Analytic Model Video Analytic operation using AI model uploaded by user	1. PrePostProcess4Decryption(.bin) 2. PrePostProcess4Decryption(.cfg) 3. PrePostProcess4Init 4. PostProcess4MakeObjResult 5. PostProcess4MakeObjResultFinalize 6. PrePostProcess4Term
251_	When it is a multi-stage inference model and the length of	It is deprecated for new implementations. It exists only for backward compatibility. For long string detection, please use model 246_.

	the string resulting from the post process is less than 35 bytes including the NULL character (Post process result type: String)	
250_	A model that replaces the first inference result in a multi-stage inference model with an ROI set directly by the user (Post process result type: Object)	1. PrePostProcess4Decryption(.bin) 2. PrePostProcess4Decryption(.cfg) 3. PrePostProcess4Init 4. PostProcess4MakeObjResultWithUyv 5. PostProcess4MakeObjResultWithUyvFinalize 6. PrePostProcess4Term
249_	When the model	1. PrePostProcess4Decryption(.bin) 2. PrePostProcess4Decryption(.cfg)

	replaces the first inference result in a multi-stage inference model with an ROI set directly by the user, and the length of the string resulting from the post-process is less than 35 bytes including the NULL character (Post-process result type: String)	3. PrePostProcess4Init 4. PostProcess4MakeStrExResultWithUyv 5. PostProcess4MakeStrExResultWithUyvFinalize 6. PrePostProcess4Term
248_	Single inference, Multi-stage inference model	1. PrePostProcess4Decryption(.bin) 2. PrePostProcess4Decryption(.cfg) 3. PrePostProcess4Init 4. PreProcessAccessOrigUyv 5. PostProcess4MakeCustomObjResult 6. PostProcess4MakeObjResultFinalize

	(Post process result type: Object) All available	7. PrePostProcess4Term
247_	Single inference, When the length of the string resulting from the post process is less than 15 bytes including the NULL character in a multi-stage inference model (Post process result type: String)	1. PrePostProcess4Decryption(.bin) 2. PrePostProcess4Decryption(.cfg) 3. PrePostProcess4Init 4. PreProcessAccessOrigUyv 5. PostProcess4MakeCustomStrResult 6. PostProcess4MakeStrResultFinalize 7. PrePostProcess4Term
246_	When it is a multi-stage inference model and	1. PrePostProcess4Decryption(.bin) 2. PrePostProcess4Decryption(.cfg) 3. PrePostProcess4Init 4. PreProcessAccessOrigUyv 5. PostProcess4MakeCustomStrExResult

	the length of the string resulting from the post process is less than 35 bytes including the NULL character (Post process result type: String)	6. PostProcess4MakeStrExResultFinalize 7. PrePostProcess4Term
--	--	--

1. Rules for naming AI models for multi-step reasoning:

ex) If it is a model called TEST that uses 5 AI models (Post process result type: String), the resolution (Width, height) of the AI model listed in the table below is an example. You can use the resolution used when training the AI model.

Model Name	Number	Width	Height	Extension	Order	Description
247_TEST	0	832	480	.bin	For stage 1 inference	For this example, the inference result returns class ids 1,2,3,4.
247_TEST	1	832	480	.bin	For 2nd stage inference	If the class id of the step 1 inference result is 1, it is used in the step 2

						inference.
247_TEST	2	832	480	.bin	For 2nd stage inference	If the class id of the step 1 inference result is 2, it is used in the step 2 inference.
247_TEST	3	832	480	.bin	For 2nd stage inference	If the class id of the step 1 inference result is 3, it is used in the step 2 inference.
247_TEST	4	832	480	.bin	For 2nd stage inference	If the class id of the step 1 inference result is 4, it is used in the step 2 inference.

2. AI model configuration for multi-stage inference: As shown in the figure below, the first AI model of multi-stage inference is number 0, and if different AI models are applied to the inference result image of the first AI model, they are distinguished by assigning numbers 1, 2, 3, 4, ...

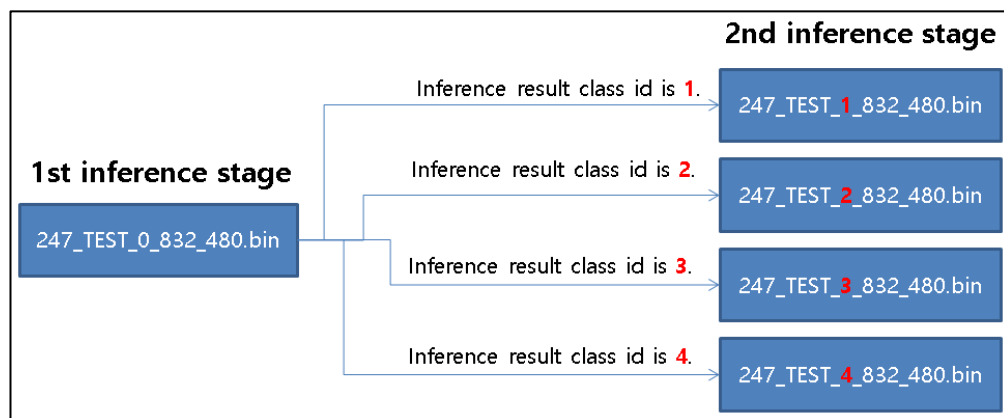


Figure 1 Multi-stage inference model numbering

Model 0 detects and crops the images to be used in 1, 2, 3, and 4.

After the first stage inference through model 0, the AI models for the second stage inference are used separately as 1, 2, 3, and 4.

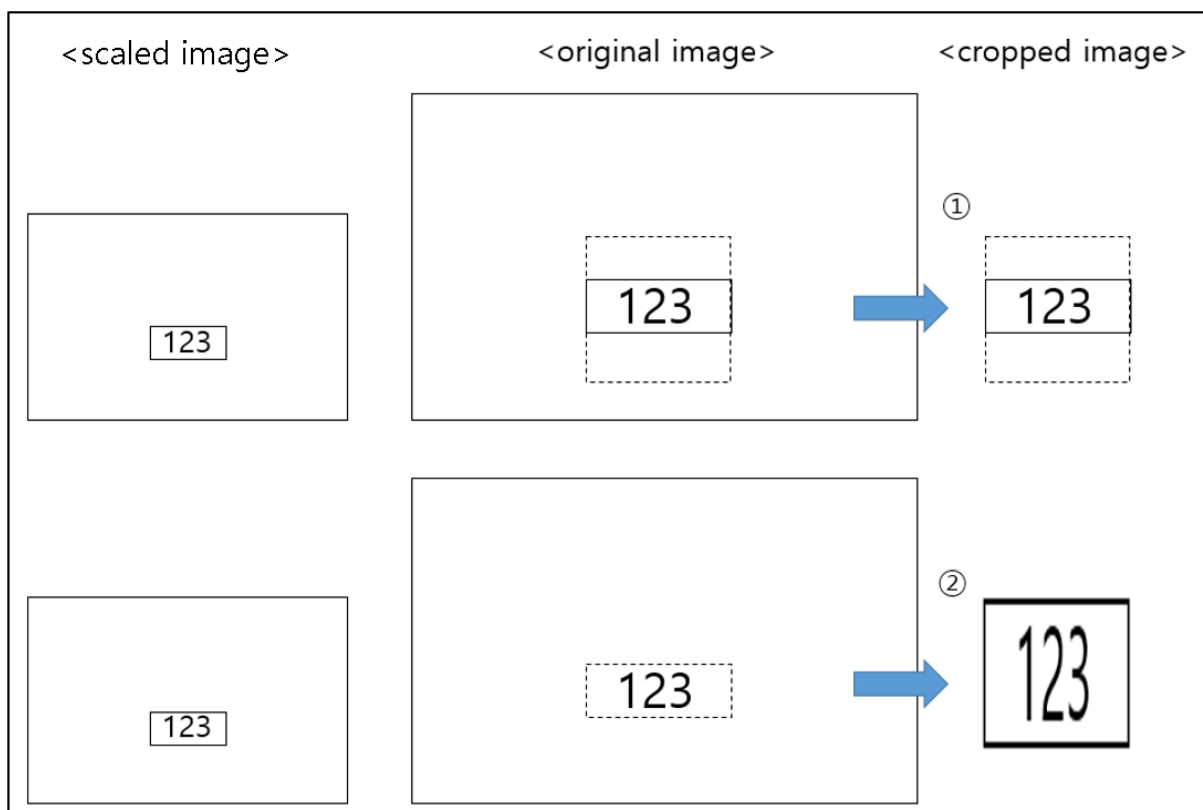
The number 5 in the 247_TEST_5_832_480.tar.gz file is the number of AI models.

The file that records the class name is prepared under the name 247_TEST.tab.

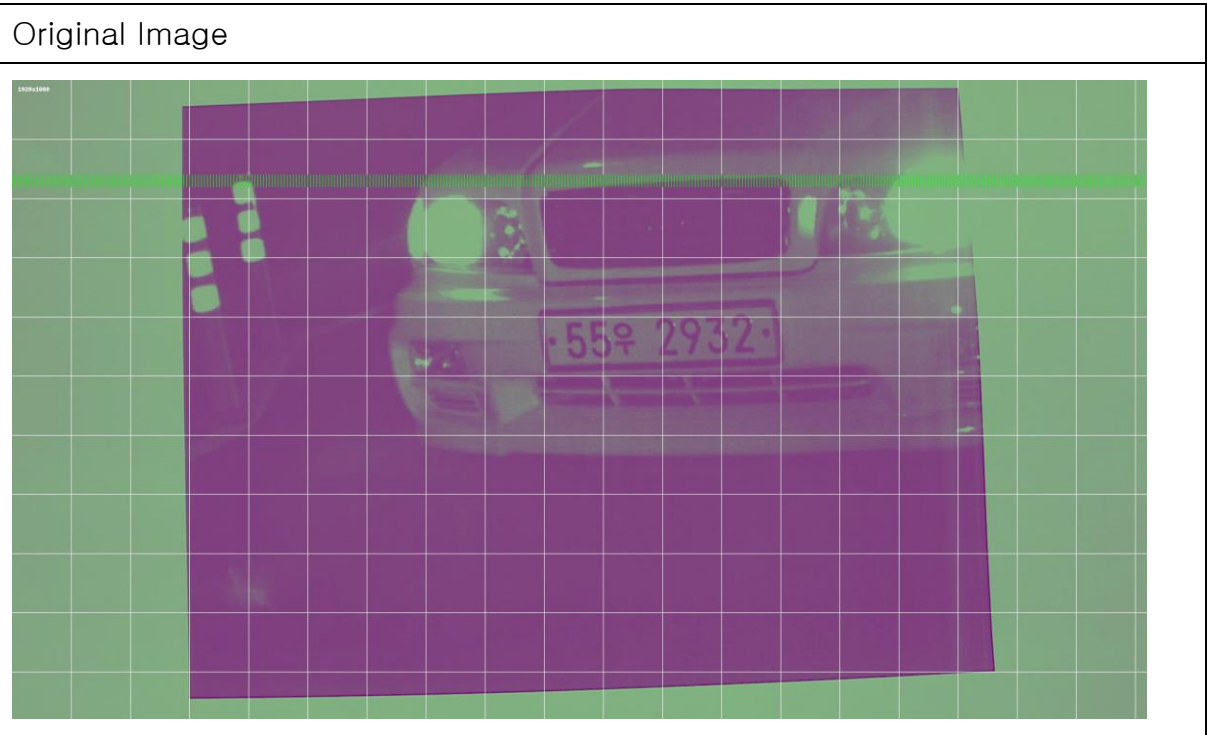
There are two ways to crop from the original image when performing multi-step inference.

As in No. 1, the method of cropping the detected object in the original image into a square shape while maintaining the aspect ratio.

As in No. 2, the method of cropping the detected object and then enlarging it into a square shape.

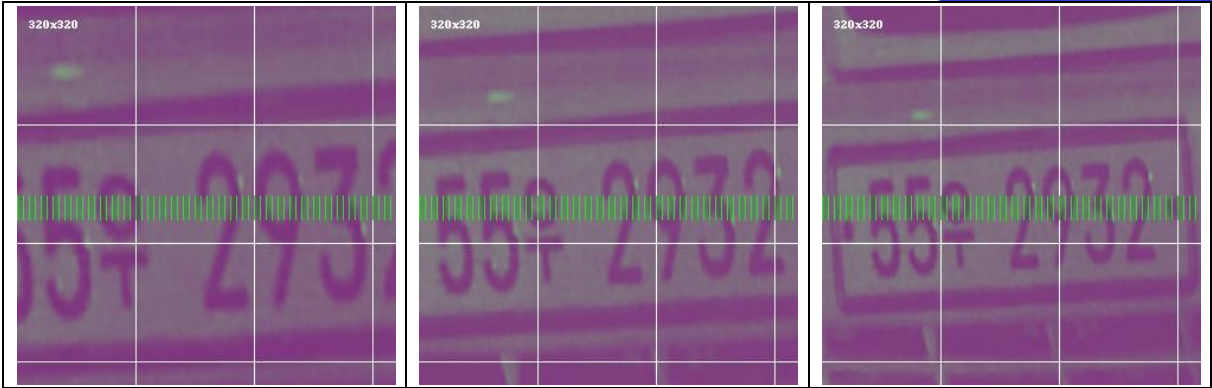


< Description of Original Image and Crop Image used in multi-stage inference >



Please note that the green vertical lines in the image below are drawn every 5 pixels to visually indicate crop offset.

Letterbox = on cropoffset = -5	Letterbox = on cropoffset = 0	Letterbox = on cropoffset = +5
-----------------------------------	----------------------------------	-----------------------------------



Letterbox = off cropoffset = -5	Letterbox = off cropoffset = 0	Letterbox = off cropoffset = +5

<How to Enable/Disable Letterbox>

FlexWATCH AI SDK

You can toggle the Letter Box option using the Letter Box Enable/Disable radio button on the AI Control page.

Object Min Confidence	40	(10 ~ 100 %)
Object Min Width	1	(1 ~ 100 %)
Object Max Width	100	(1 ~ 100 %)
Object Min Height	1	(1 ~ 100 %)
Object Max Height	100	(1 ~ 100 %)
Object Tracking Enable	☒ Enable ☐ Disable	
Objects Display on Raw Video	☐ Enable ☒ Disable	
Objects Display on Viewer	☒ Enable ☐ Disable	
Synchronize video and BBox	☐ Enable ☒ Disable	
Letter Box Enable/Disable	☒ Enable ☐ Disable	
Crop Offset	5	(-100~100)
Crop from the copied original UYV	☐ Enable ☒ Disable	

<How to set Crop Offset>

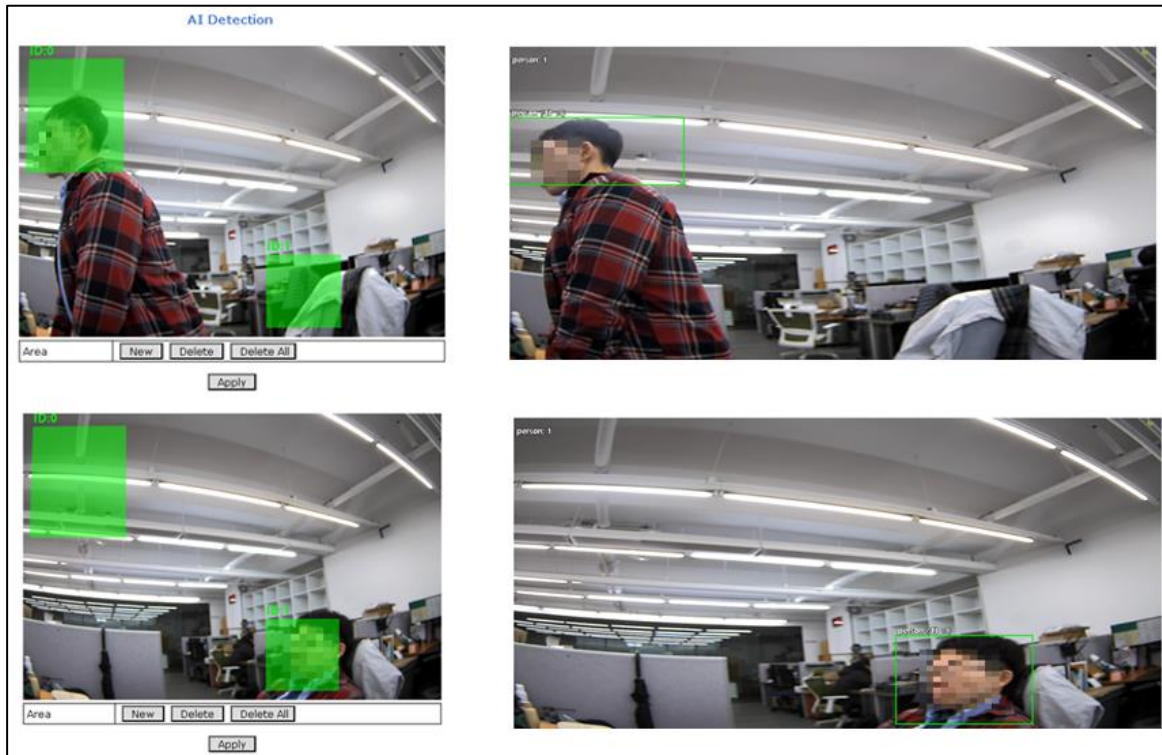
You can enter the offset value in the Crop Offset edit box on the AI Control page and apply it.

Object Min Confidence	40	(10 ~ 100 %)
Object Min Width	1	(1 ~ 100 %)
Object Max Width	100	(1 ~ 100 %)
Object Min Height	1	(1 ~ 100 %)
Object Max Height	100	(1 ~ 100 %)
Object Tracking Enable	☒ Enable ☐ Disable	
Objects Display on Raw Video	☐ Enable ☒ Disable	
Objects Display on Viewer	☒ Enable ☐ Disable	
Synchronize video and BBox	☐ Enable ☒ Disable	
Letter Box Enable/Disable	☒ Enable ☐ Disable	
Crop Offset	5	(-100~100)
Crop from the copied original UYV	☐ Enable ☒ Disable	

Apply

An example of a model that replaces the first inference result in a multi-stage inference model with a ROI set by the user:

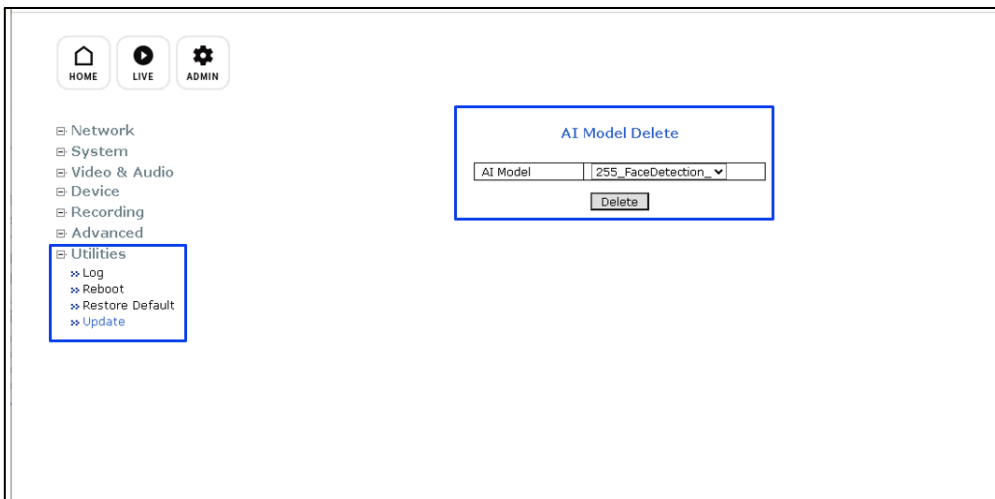
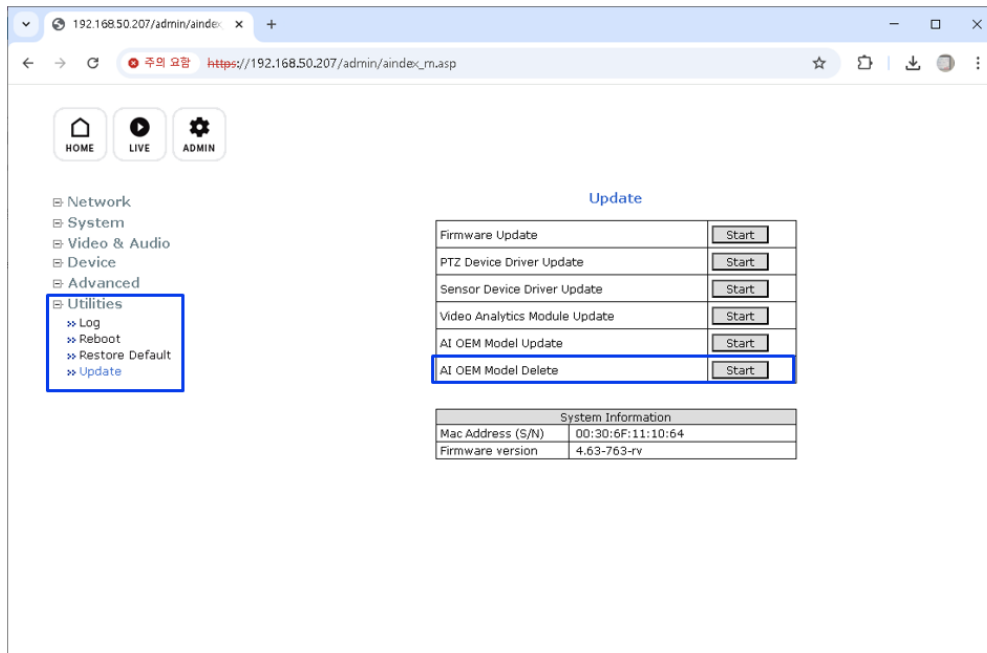
In the Web ADMIN -> Video & Audio -> AI Control menu, specify the ROI in the AI Detection item. You can specify multiple ROIs. In the screen below, the Person detection model was used for testing purposes.



1.14. How to delete AI OEM Model

EN675 Because the camera's storage space is limited, many models cannot be uploaded and used at the same time. When the camera's storage space is insufficient, the AI models uploaded by the user can be deleted using the following method. Also, when updating the firmware file for a camera firmware update, if the storage space is insufficient, the AI model files must be deleted. The AI model currently in use cannot be deleted, so to delete it, change it to 000_NotSelected and then delete it.

Webadmin -> Utilities -> Update -> AI OEM Model Delete



1.15. Download and build the FlexWatch AI SDK

Basic post processing for Object Detection is provided internally.

If the user wants, a separate post processing algorithm can be implemented. Please refer to the sample code distributed with the SDK.

The latest AI SDK supports backward compatibility, but for various PrePostProcess functions and performance improvements, please implement the AI OEM model using

FlexWATCH AI SDK

the latest firmware and the latest AI SDK.

Camera firmware version and recommended FlexWatch AI SDK version

FlexWatch AI SDK version	Camera firmware version
4.1.0	The version prior to rf_ec-4.63-759-rv.tar.gz
4.1.1	The version after rf_ec-4.63-760-rv.tar.gz

1. Prepare Ubuntu PC (18.04 or 20.04)
2. Download FlexWatch AI SDK

FlexWatch_AI_SDK_x.x.x.zip을 다운로드 받습니다.

URL : sdk.flexwatch.com

ID	fwrnd
PASSWORD	flexwatch

3. Install required packages

```
$ sudo apt-get update
$ sudo apt-get install build-essential bison flex device-tree-compiler libncurses-dev
```

4. Install the cross compilation toolchain

```
$ unzip FlexWatch_AI_SDK_x.x.x.x.zip
```

When unzipped, you will get en675_linux_cross.tgz, en675_trainer_sy.tar, lib.tar.gz.

Install the cross compile environment by executing the following command in the unzipped folder.

```
$ tar xzf en675_linux_cross.tgz
$ cd en675_linux_cross
```

```
$ chmod 777 en675_linux_cross_install.sh↓  
$ sudo ./en675_linux_cross_install.sh↓
```

5. Unpack and build the SDK

Build libPrePostProcess.so.1.0.0 by executing the following command in the folder where you unzipped the FlexWatch_AI_SDK_x.x.x.x.zip file. (If you need to modify the source file, refer to Section 1.17 of this manual for how to do so.)

```
$ tar -xzf lib_yyyymmdd.tar.gz  
$ cd lib  
$ make clean  
$ make
```

6. Verify the built .so.1.0.0 and grant execution permission

```
/lib$ ll libPrePostProcess.so*  
-rwxrwxr-x 1 yhlee yhlee 20816  8 23 11:16 libPrePostProcess.so.1.0.0*  
$ chmod +x libPrePostProcess.so.1.0.0
```

7. The built libPrePostProcess.so.1.0.0 is used when creating the AI OEM Model in section 1.12.1 of this manual.

1.16. Create an AI OEM MODEL tar.gz file for multi-step inference

Once the AI model for multi-stage inference and the libPrePostProcess.so.1.0.0 file are prepared, create the AI OEM model tar.gz file using the command below.

(The .tab file is commonly used by the secondary models corresponding to 1, 2, 3, and 4, and is prepared by referring to 1.8.3 of this manual.)

tar.gz is compressed on the Linux OS.

If the post process result type is String:

```
tar -zcvf 247_TEST_5_832_480.tar.gz \
```

```
247_TEST_0_832_480.bin \  
247_TEST_1_832_480.bin \  
247_TEST_2_832_480.bin \  
247_TEST_3_832_480.bin \  
247_TEST_4_832_480.bin \  
247_TEST_0_832_480.cfg \  
247_TEST_1_832_480.cfg \  
247_TEST_2_832_480.cfg \  
247_TEST_3_832_480.cfg \  
247_TEST_4_832_480.cfg \  
247_TEST.tab \  
libPrePostProcess.so.1.0.0
```

If the post process result type is Object:

```
tar -zcvf 248_TEST_5_832_480.tar.gz \  
248_TEST_0_832_480.bin \  
248_TEST_1_832_480.bin \  
248_TEST_2_832_480.bin \  
248_TEST_3_832_480.bin \  
248_TEST_4_832_480.bin \  
248_TEST_0_832_480.cfg \  
248_TEST_1_832_480.cfg \  
248_TEST_2_832_480.cfg \  
248_TEST_3_832_480.cfg \  
248_TEST_4_832_480.cfg \  
248_TEST.tab \  
libPrePostProcess.so.1.0.0
```

If the model replaces the first inference result in the multi-step inference model

FlexWATCH AI SDK

with an ROI that the user sets directly and the post-process result type is Object: Due to the naming convention, a dummy model is required. In the example below, you can use 250_TEST_1_832_480.bin, 250_TEST_1_832_480.cfg by duplicating them to create 250_TEST_0_832_480.bin, 250_TEST_0_832_480.cfg. (A model that replaces the first inference result in the multi-step inference model with an ROI that the user sets directly requires a GUI implementation related to the ROI type definition.)

```
tar -zcvf 250_TEST_2_832_480.tar.gz \  
250_TEST_0_832_480.bin \  
250_TEST_1_832_480.bin \  
250_TEST_0_832_480.cfg \  
250_TEST_1_832_480.cfg \  
250_TEST.tab \  
libPrePostProcess.so.1.0.0
```

If the model replaces the first inference result in the multi-step inference model with an ROI set directly by the user and the post-process result type is a String of 35 bytes or less:

Due to the naming convention, a dummy model is required. In the example below, you can use 249_TEST_1_832_480.bin, 249_TEST_1_832_480.cfg by duplicating them to create 249_TEST_0_832_480.bin, 249_TEST_0_832_480.cfg. (A model that replaces the first inference result in the multi-step inference model with an ROI set directly by the user requires a GUI implementation related to the ROI type definition.)

```
tar -zcvf 249_TEST_2_832_480.tar.gz \  
249_TEST_0_832_480.bin \  
249_TEST_1_832_480.bin \  
249_TEST_0_832_480.cfg \  
249_TEST_1_832_480.cfg \  

```

```
249_TEST.tab \  
libPrePostProcess.so.1.0.0
```

The created AI OEM model can be uploaded to the IP camera and used. (Refer to 1.8.5~6)

1.17. How to edit Pre Post Processing source file

Basic post processing (object tracking, Kalman filter) is provided internally. If the user wants, a separate post processing algorithm can be implemented. Please refer to the sample code distributed with the SDK. Below is a simple function description.

1. If you need to encrypt and decrypt your own AI model to ensure its security, you can perform decryption by referring to '1.20. Decrypt function for encrypted custom AI model file' in this manual.
2. In the PrePostProcess4Init function, check the license and allocate the resources required to implement the Post Processing algorithm. Enter the user's license key in the UserSecretKey array. (Do not disclose UserSecretKey.)
3. Implement the Post Processing algorithm that suits your AI model in the PostProcess4MakeStrResult function.

The g_bLicenseSucceed variable is used to check whether the license key matches, and if the license key matches, the pObj variable, which is the inference result, is received as input, post-processed, and the post-processing result is received through the pResult variable.

4. There may be multiple calls to the PostProcess4MakeStrResult function depending on the number of objects detected in an image. The PostProcess4MakeStrResultFinalize function is called only once when all PostProcess4MakeStrResult function calls for a frame are completed.
5. Implement the Post Processing algorithm that suits your AI model in the PostProcess4MakeStrExResult function.

The g_bLicenseSucceed variable is used to check whether the license key matches, and if the license key matches, the pObj variable, which is the inference result, is received as input, post-processed, and the post-processing result is received through the pResult variable.

6. There may be multiple PostProcess4MakeStrExResult function calls depending on the number of objects detected in an image. The PostProcess4MakeStrExResultFinalize function is called only once when all PostProcess4MakeStrExResult function calls for a frame are completed.
7. The PreProcessAccessOrigUyv function is called only when the multi-step inference is performed and the return value of PreProcessIsNeedAccessOrigUyv() is 1, and the original UYV image can be accessed.
8. Implement the Post Processing algorithm that suits your AI model in the PostProcess4MakeObjResult function.

The g_bLicenseSucceed variable is used to check whether the license key matches, and if the license key matches, the pObj variable, which is the inference result, is received as input, post-processed, and the post-processing result is received through the pResult variable.

9. There may be multiple PostProcess4MakeObjResult function calls depending on the number of objects detected in an image. The PostProcess4MakeObjResultFinalize function is called only once, when all PostProcess4MakeObjResult function calls for a frame are completed.
10. If there are resources (memory, handles, etc.) allocated by the user for implementing the Post Processing algorithm in the PrePostProcess4Init function, add code to return the resources to the PrePostProcess4Term function to prevent memory leaks.
11. You can test whether so works by uploading the TestPostProcess program built with libPrePostProcess.so.1.0.0 to the camera via telnet. For information on how to connect via telnet, see Appendix 1.14.1.
12. Implement the Post Processing algorithm that suits your AI model in the PostProcess4MakeObjResultWithUyv function.

The `g_bLicenseSucceed` variable is used to check whether the license key matches, and if the license key matches, the `pObj` variable, which is the inference result, is received as input, post-processed, and the post-processing result is received through the `pResult` variable.

13. There may be multiple calls to the `PostProcess4MakeObjResultWithUyv` function depending on the number of objects detected in an image. The `PostProcess4MakeObjResultWithUyvFinalize` function is called only once when all `PostProcess4MakeObjResultWithUyv` function calls for a frame are completed.

14. Implement the Post Processing algorithm that suits your AI model in the `PostProcess4MakeStrExResultWithUyv` function.

The `g_bLicenseSucceed` variable is used to check whether the license key matches, and if the license key matches, the `pObj` variable, which is the inference result, is received as input, post-processed, and the post-processing result is received through the `pResult` variable.

15. There may be multiple calls to the `PostProcess4MakeStrExResultWithUyv` function depending on the number of objects detected in an image. The `PostProcess4MakeStrExResultWithUyvFinalize` function is called only once when all `PostProcess4MakeStrExResultWithUyv` function calls for a frame are completed.

16. If there are resources (memory, handles, etc.) allocated by the user for implementing the Post Processing algorithm in the `PrePostProcess4Init` function, add code to return the resources to the `PrePostProcess4Term` function to prevent memory leaks.

17. You can test whether so works by uploading the `TestPostProcess` program built with `libPrePostProcess.so.1.0.0` to the camera via telnet. For information on how to connect via telnet, see Appendix 1.14.1.

18. Implement the Post Processing algorithm that fits your AI model in the `PostProcess4MakeCustomObjResult` function. There may be multiple

PostProcess4MakeCustomObjResult function calls depending on the number of objects detected in an image. The PostProcess4MakeObjResultFinalize function is called only once when all PostProcess4MakeCustomObjResult function calls for a frame are completed.

19. Implement the Post Processing algorithm that fits your AI model in the PostProcess4MakeCustomStrResult function. There may be multiple PostProcess4MakeCustomStrResult function calls depending on the number of objects detected in an image. The PostProcess4MakeStrResultFinalize function is called only once when all PostProcess4MakeCustomStrResult function calls for a frame are completed.
20. Implement the Post Processing algorithm that fits your AI model in the PostProcess4MakeCustomStrExResult function. There may be multiple PostProcess4MakeCustomStrExResult function calls depending on the number of objects detected in an image. The PostProcess4MakeStrExResultFinalize function is called only once when all PostProcess4MakeCustomStrExResult function calls for a frame are completed.

1.18. How to check AI inference and Post Process logs

Here's how to check AI inference and post-process logs:

1. Upload the AI OEM model by referring to 1.8.5. (If the model has already been uploaded, you can skip this step.)
2. Refer to 1.8.6 and select the AI model as shown below and click the 'apply' button.
3. Connect to the camera via telnet. (Refer to Appendix 1.14.1 Telnet Connection Method)
4. After connecting to the camera via telnet, you can check the log by executing the command below in the shell.

```
$/etc/init.d/fweyenixai restart
```

Log output example)

```
int PostProcess4MakeStrResult(int AiModelId, INFERENCE_OBJ_T * pObj, POST_PROCESS_STR_RESULT_T * pResult)
{
    int nIdx      = 0;
    int nXmin     = 255;
    int nXmax     = -1;
    int nYmin     = 255;
    int nYmax     = -1;
    int nConfiSum = 0;

    // if (!g_bLicenseSucceed)
    // return -1;
    fprintf(stderr, "%s%d, AiModelId : %d\n", __FUNCTION__, __LINE__, AiModelId);
    for (nIdx = 0; nIdx < pObj->cnt; nIdx++)
    {
        if (0 == pObj->obj[nIdx].class)
            continue;

        if (SAMPLE_CLASS_ID_CNT <= pObj->obj[nIdx].class)
            continue;

        if (STR_RESULT_MAX_LEN <= (strlen(pResult->pString))
            continue;

        strcat(pResult->pString, g_pClassName[pObj->obj[nIdx].class]);
        nXmin = (nXmin < pObj->obj[nIdx].x_min)?nXmin:pObj->obj[nIdx].x_min;
        nXmax = (nXmax > pObj->obj[nIdx].x_max)?nXmax:pObj->obj[nIdx].x_max;
        nYmin = (nYmin < pObj->obj[nIdx].y_min)?nYmin:pObj->obj[nIdx].y_min;
        nYmax = (nYmax > pObj->obj[nIdx].y_max)?nYmax:pObj->obj[nIdx].y_max;
        nConfiSum += pObj->obj[nIdx].score;
    }
}
```

1.19. Korean font support through osd

The Korean fonts that can be displayed in raw video are as shown in the table below.

ㄱ	가, 강, 거, 경, 고, 광, 구, 기
ㄴ	나, 남, 너, 노, 누
ㄷ	다, 대, 더, 도, 두
ㄹ	라, 러, 로, 루
ㅁ	마, 머, 모, 무
ㅂ	바, 배, 버, 보, 부, 북
ㅅ	사, 산, 서, 세, 소, 수
ㅇ	아, 어, 오, 우, 울, 원, 인
ㅈ	자, 저, 제, 조, 중, 주, 전
ㅊ	차, 천, 초, 충
ㅋ	카, 코
ㅌ	타
ㅍ	파
ㅎ	하, 허, 호

1.20. UTF-8 code table for supported Korean osd

The UTF-8 codes corresponding to the supported Korean characters are as shown in the table below.

Korean	utf-8 code
어	0xEC, 0x96, 0xB4
오	0xEC, 0x98, 0xA4
우	0xEC, 0x9A, 0xB0

버	0xEB, 0xB2, 0x84
보	0xEB, 0xB3, 0xB4
부	0xEB, 0xB6, 0x80
다	0xEB, 0x8B, 0xA4
더	0xEB, 0x8D, 0x94
도	0xEB, 0x8F, 0x84
두	0xEB, 0x91, 0x90
하	0xED, 0x95, 0x98
허	0xED, 0x97, 0x88
호	0xED, 0x98, 0xB8
저	0xEC, 0xA0, 0x80
조	0xEC, 0xA1, 0xB0
주	0xEC, 0xA3, 0xBC
가	0xEA, 0xB0, 0x80
거	0xEA, 0xB1, 0xB0
고	0xEA, 0xB3, 0xA0

구	0xEA, 0xB5, 0xAC
라	0xEB, 0x9D, 0xBC
러	0xEB, 0x9F, 0xAC
로	0xEB, 0xA1, 0x9C
루	0xEB, 0xA3, 0xA8
마	0xEB, 0xA7, 0x88
머	0xEB, 0xA8, 0xB8
모	0xEB, 0xAA, 0xA8
무	0xEB, 0xAC, 0xB4
나	0xEB, 0x82, 0x98
너	0xEB, 0x84, 0x88
노	0xEB, 0x85, 0xB8
누	0xEB, 0x88, 0x84
서	0xEC, 0x84, 0x9C
소	0xEC, 0x86, 0x8C
수	0xEC, 0x88, 0x98

바	0xEB, 0xB0, 0x94
사	0xEC, 0x82, 0xAC
아	0xEC, 0x95, 0x84
자	0xEC, 0x9E, 0x90
배	0xEB, 0xB0, 0xB0
울	0xEC, 0x9A, 0xB8
산	0xEC, 0x82, 0xB0
인	0xEC, 0x9D, 0xB8
천	0xEC, 0xB2, 0x9C
대	0xEB, 0x8C, 0x80
광	0xEA, 0xB4, 0x91
전	0xEC, 0xA0, 0x84
경	0xEA, 0xB2, 0xBD
기	0xEA, 0xB8, 0xB0
강	0xEA, 0xB0, 0x95
원	0xEC, 0x9B, 0x90

총	0xEC, 0xB6, 0xA9
북	0xEB, 0xB6, 0x81
남	0xEB, 0x82, 0xA8
제	0xEC, 0xA0, 0x9C
세	0xEC, 0x84, 0xB8
종	0xEC, 0xA2, 0x85
카	0xEC, 0xB9, 0xB4
타	0xED, 0x83, 0x80
파	0xED, 0x8C, 0x8C
차	0xEC, 0xB0, 0xA8
초	0xEC, 0xB4, 0x88
코	0xEC, 0xBD, 0x94

1.21. Decrypt function for encrypted custom ai model file

Here is the function to decrypt an encrypted user-defined AI model.

Encryption of the model is not required. You can use it when necessary.

It supports encryption and decryption of .bin, .cfg files, but does not support encryption of .tab files.

This is because the .tab file is required when selecting a class on the AI Control page.

FlexWATCH AI SDK

You can use the function below in PostProcessTemplate.cpp to implement decryption and use it.

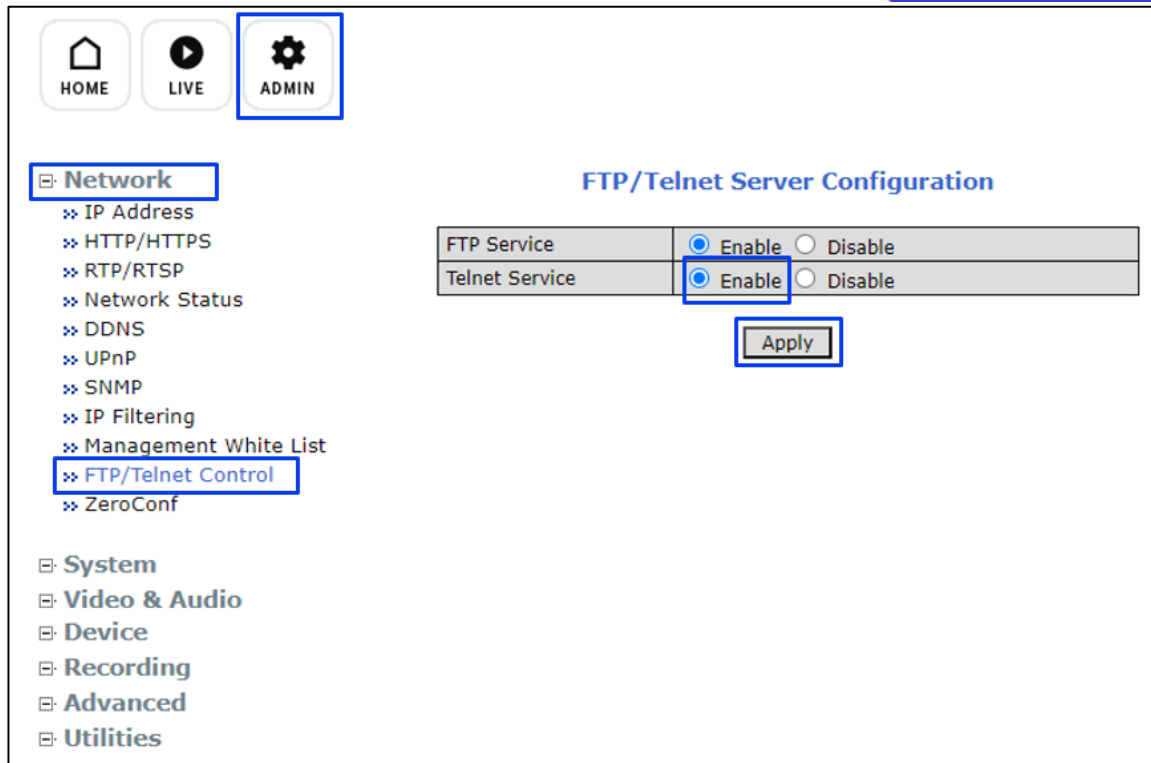
```
int    PrePostProcess4Decryption(char * pEncFilePath, const char * pDecFilePath)
{
//      If the encrypted file is successfully decrypted and stored in pDecFilePath, it
//      should return 0.
//      return 0;

//      Returns -1 if no action is taken or decryption fails.
//      return -1;
}
```

1.22. Appendix

1. Telnet connection method

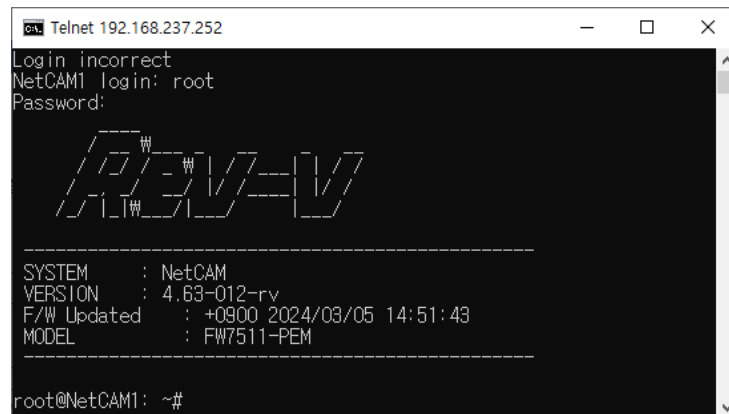
Admin -> Network -> FTP/Telnet Control Enable Telnet Service from the menu



Open terminal and type the following:

```
telnet -a <camera ip address>
```

When you enter your ID/PW, a shell will appear as shown below. You can execute the necessary commands in this shell.



2. Introduction to the labeling tool

Program Name	labelImg
Download URL	https://github.com/HumanSignal/labelImg/releases/tag/v1.8.1

3. Introduction to Yolo Label (.txt) to Pascal VOC type (.xml) conversion tool

Program Name	convert-yolo-to-pascalvoc
Download URL	https://github.com/carolinepacheco/convert-yolo-to-pascalvoc/tree/master

Activate the virtual environment created in 1.2 and install additional packages with the command below.

```
$pip install lxml
```

You can convert yolo type labels to pascalvoc type labels by referring to the notes in the download URL. To use the converted label file for training, refer to the structure of the PASCAL_VOC_Dataset/PASCAL_VOC_2012 folder downloaded in 1.4.1.

1.23. Document Revision History

Version	Date	Description	Modified by
T4048-8	2024.11.12	Specify the PrePostProcess function name and function call order called according to the prefix (number) of the OEM AI Model.	YH LEE
T4048-9	2024.12.18	Added description for a model that	YH LEE

		replaces the first inference result in a multi-stage inference model with an ROI that the user sets themselves	
T4048-10	2025.01.20	1. Add a model whose Post Process result type is String less than 35 bytes to replace the first inference result in the multi-step inference model with the ROI set by the user. 2. Add a PreProcessIsNeedAccessOrigUyv() function to provide an option to access and use the Original UYV image only when necessary.	YH LEE
T4048-11	2025.04.17	1. Added method to delete AI OEM Model 2. Modified sentence (file name, expression) 3. Updated web UI image	YH LEE
T4048-12	2025.04.19	1. Added method to delete AI OEM Model 2. Modified sentence (file name, expression) 3. Updated web UI image based on latest firmware 4. Changed PrePostProcess function name	YH LEE



T4048-13	2025.04.30	Change SDK download URL	YHLEE
----------	------------	-------------------------	-------